My Project

Generated by Doxygen 1.15.0

1 Class Index 1

1.1 Class List . . . 1

2 File Index 3

2.1 File List . . . 3

3 Class Documentation 5

3.1 backcmd Struct Reference . . . 5
3.2 buf Struct Reference . . . 6
3.3 cmd Struct Reference . . . 7
3.4 context Struct Reference . . . 7
3.5 cpu Struct Reference . . . 8
3.6 devsw Struct Reference . . . 9
3.7 dinode Struct Reference . . . 9
3.8 dirent Struct Reference . . . 9
3.9 elfhdr Struct Reference . . . 10
3.10 execcmd Struct Reference . . . 10
3.11 file Struct Reference . . . 11
3.12 gatedesc Struct Reference . . . 12
3.13 header Union Reference . . . 12
3.14 inode Struct Reference . . . 13
3.15 ioapic Struct Reference . . . 14
3.16 listcmd Struct Reference . . . 14
3.17 log Struct Reference . . . 15
3.18 logheader Struct Reference . . . 15
3.19 mp Struct Reference . . . 16
3.20 mpconf Struct Reference . . . 16
3.21 mpioapic Struct Reference . . . 16
3.22 mpproc Struct Reference . . . 17
3.23 pipe Struct Reference . . . 17
3.24 pipecmd Struct Reference . . . 18
3.25 proc Struct Reference . . . 19
3.26 proghdr Struct Reference . . . 20
3.27 redircmd Struct Reference . . . 20
3.28 rtcdate Struct Reference . . . 21
3.29 run Struct Reference . . . 21
3.30 segdesc Struct Reference . . . 22
3.31 sleeplock Struct Reference . . . 23
3.32 spinlock Struct Reference . . . 24
3.33 stat Struct Reference . . . 24
3.34 superblock Struct Reference . . . 25
3.35 taskstate Struct Reference . . . 25
3.36 trapframe Struct Reference . . . 26

4 File Documentation **27**

4.1 asm.h . . . 27
4.2 buf.h . . . 27
4.3 date.h . . . 27
4.4 defs.h . . . 28
4.5 elf.h . . . 30
4.6 fcntl.h . . . 30
4.7 file.h . . . 30
4.8 fs.h . . . 31
4.9 kbd.h . . . 32
4.10 memlayout.h . . . 33
4.11 mmu.h . . . 33
4.12 mp.h . . . 36
4.13 param.h . . . 36
4.14 proc.h . . . 36
4.15 sleeplock.h . . . 37
4.16 spinlock.h . . . 37
4.17 stat.h . . . 38
4.18 syscall.h . . . 38
4.19 traps.h . . . 38
4.20 types.h . . . 39
4.21 user.h . . . 39
4.22 x86.h . . . 39

Index **43**

Chapter 1

Class Index

1.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

backcmd . . . 5
buf . . . 6
cmd . . . 7
context . . . 7
cpu . . . 8
devsw . . . 9
dinode . . . 9
dirent . . . 9
elfhdr . . . 10
execcmd . . . 10
file . . . 11
gatedesc . . . 12
header . . . 12
inode . . . 13
ioapic . . . 14
listcmd . . . 14
log . . . 15
logheader . . . 15
mp . . . 16
mpconf . . . 16
mpioapic . . . 16
mpproc . . . 17
pipe . . . 17
pipecmd . . . 18
proc . . . 19
proghdr . . . 20
redircmd . . . 20
rtcdate . . . 21
run . . . 21
segdesc . . . 22
sleeplock . . . 23
spinlock . . . 24
stat . . . 24
superblock . . . 25
taskstate . . . 25
trapframe . . . 26

Chapter 2

File Index

2.1 File List

Here is a list of all documented files with brief descriptions:

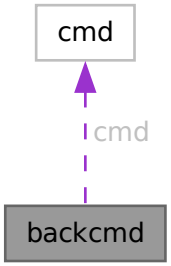
asm.h . . . 27
buf.h . . . 27
date.h . . . 27
defs.h . . . 28
elf.h . . . 30
fcntl.h . . . 30
file.h . . . 30
fs.h . . . 31
kbd.h . . . 32
memlayout.h . . . 33
mmu.h . . . 33
mp.h . . . 36
param.h . . . 36
proc.h . . . 36
sleeplock.h . . . 37
spinlock.h . . . 37
stat.h . . . 38
syscall.h . . . 38
traps.h . . . 38
types.h . . . 39
user.h . . . 39
x86.h . . . 39

Chapter 3

Class Documentation

3.1 backcmd Struct Reference

Collaboration diagram for backcmd:



Public Attributes

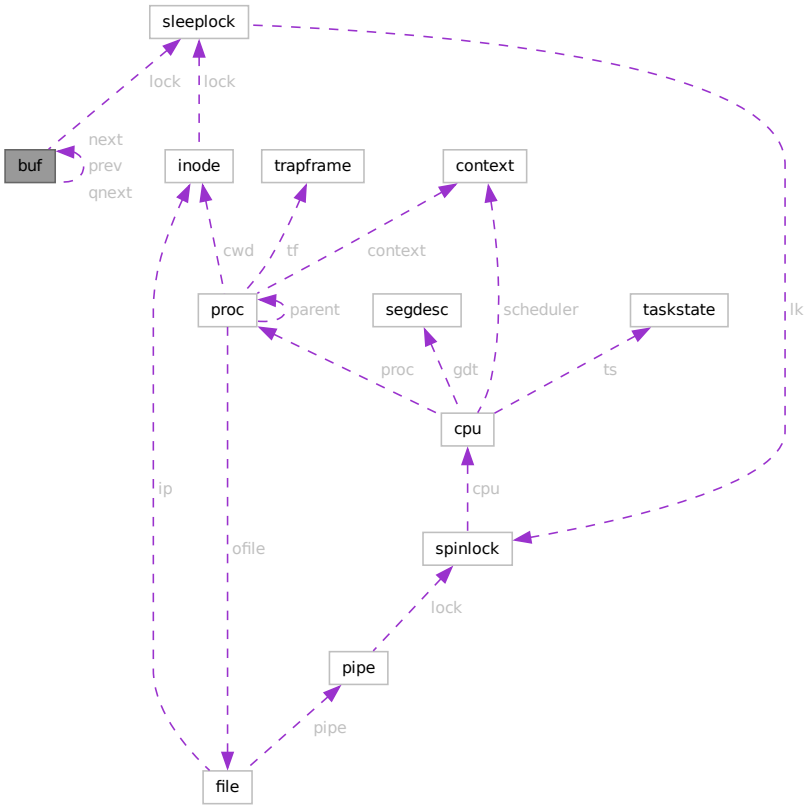
- int **type**
- struct cmd * **cmd**

The documentation for this struct was generated from the following file:

- sh.c

3.2 buf Struct Reference

Collaboration diagram for buf:



Public Attributes

- int **flags**
- uint **dev**
- uint **blockno**
- struct sleeplock **lock**
- uint **refcnt**
- struct buf * **prev**
- struct buf * **next**
- struct buf * **qnext**
- uchar **data** [BSIZE]

The documentation for this struct was generated from the following file:

- buf.h

3.3 cmd Struct Reference

Public Attributes

- int **type**

The documentation for this struct was generated from the following file:

- sh.c

3.4 context Struct Reference

Public Attributes

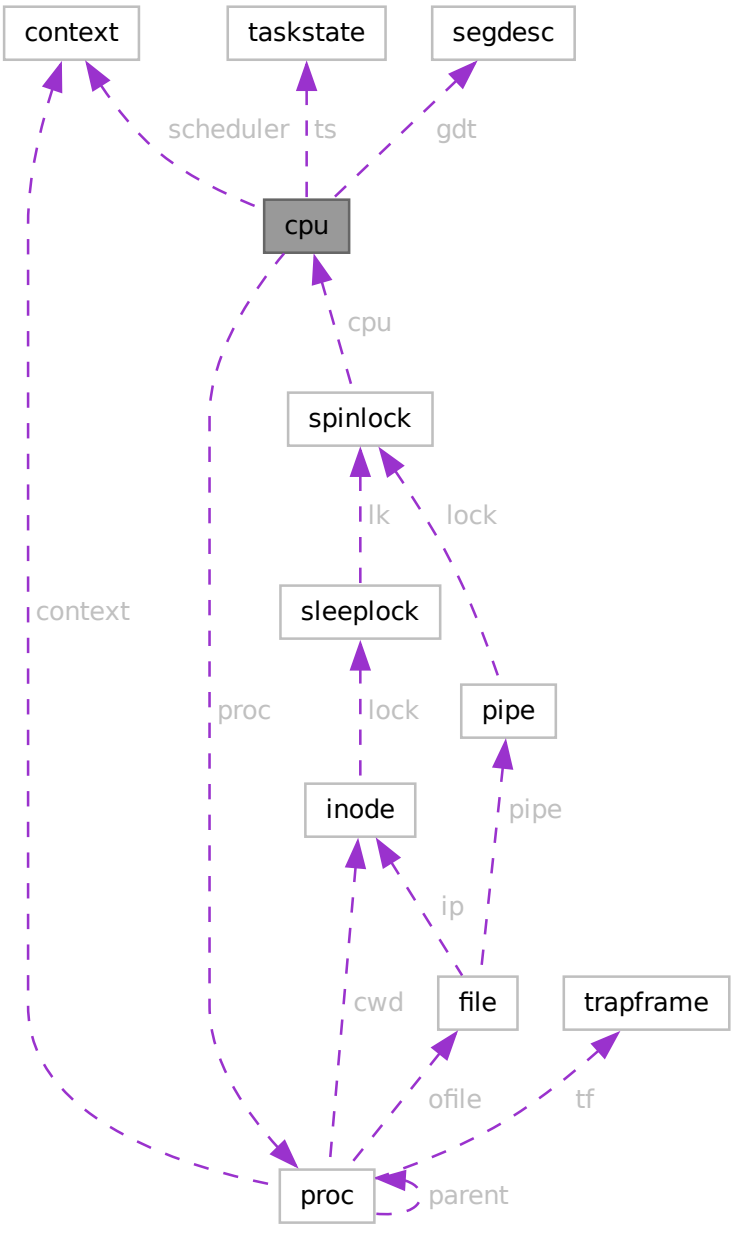
- uint **edi**
- uint **esi**
- uint **ebx**
- uint **ebp**
- uint **eip**

The documentation for this struct was generated from the following file:

- proc.h

3.5 cpu Struct Reference

Collaboration diagram for cpu:



Public Attributes

- uchar **apicid**
- struct context * **scheduler**
- struct taskstate **ts**

- struct segdesc **gdt** [NSEGS]
- volatile uint **started**
- int **ncli**
- int **intena**
- struct proc * **proc**

The documentation for this struct was generated from the following file:

- proc.h

3.6 devsw Struct Reference

Public Attributes

- int(* **read**)(struct inode *, char *, int)
- int(* **write**)(struct inode *, char *, int)

The documentation for this struct was generated from the following file:

- file.h

3.7 dinode Struct Reference

Public Attributes

- short **type**
- short **major**
- short **minor**
- short **nlink**
- uint **size**
- uint **addrs** [NDIRECT+1]

The documentation for this struct was generated from the following file:

- fs.h

3.8 dirent Struct Reference

Public Attributes

- ushort **inum**
- char **name** [DIRSIZ]

The documentation for this struct was generated from the following file:

- fs.h

3.9 elfhdr Struct Reference

Public Attributes

- uint **magic**
- uchar **elf** [12]
- ushort **type**
- ushort **machine**
- uint **version**
- uint **entry**
- uint **phoff**
- uint **shoff**
- uint **flags**
- ushort **ehsize**
- ushort **phentsize**
- ushort **phnum**
- ushort **shentsize**
- ushort **shnum**
- ushort **shstrndx**

The documentation for this struct was generated from the following file:

- elf.h

3.10 execcmd Struct Reference

Public Attributes

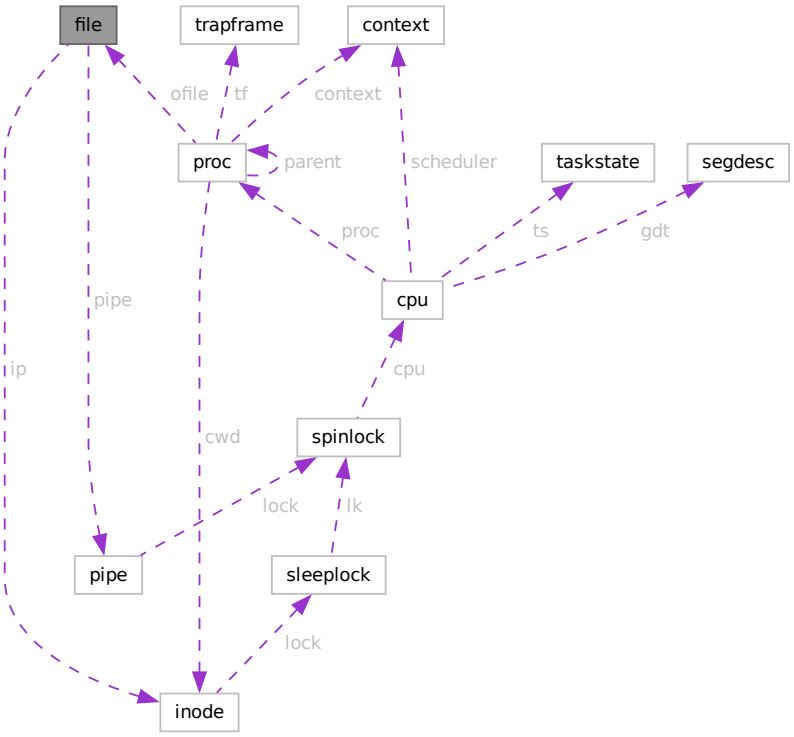
- int **type**
- char * **argv** [MAXARGS]
- char * **eargv** [MAXARGS]

The documentation for this struct was generated from the following file:

- sh.c

3.11 file Struct Reference

Collaboration diagram for file:



Public Types

- enum { **FD_NONE** , **FD_PIPE** , **FD_INODE** }

Public Attributes

- enum file:: { ... } **type**
- int **ref**
- char **readable**
- char **writable**
- struct pipe * **pipe**
- struct inode * **ip**
- uint **off**

The documentation for this struct was generated from the following file:

- file.h

3.12 gatedesc Struct Reference

Public Attributes

- uint **off_15_0**: 16
- uint **cs**: 16
- uint **args**: 5
- uint **rsv1**: 3
- uint **type**: 4
- uint **s**: 1
- uint **dpl**: 2
- uint **p**: 1
- uint **off_31_16**: 16

The documentation for this struct was generated from the following file:

- mmu.h

3.13 header Union Reference

Collaboration diagram for header:



Public Attributes

- struct {
  union header * **ptr**
  uint **size**
  } **s**

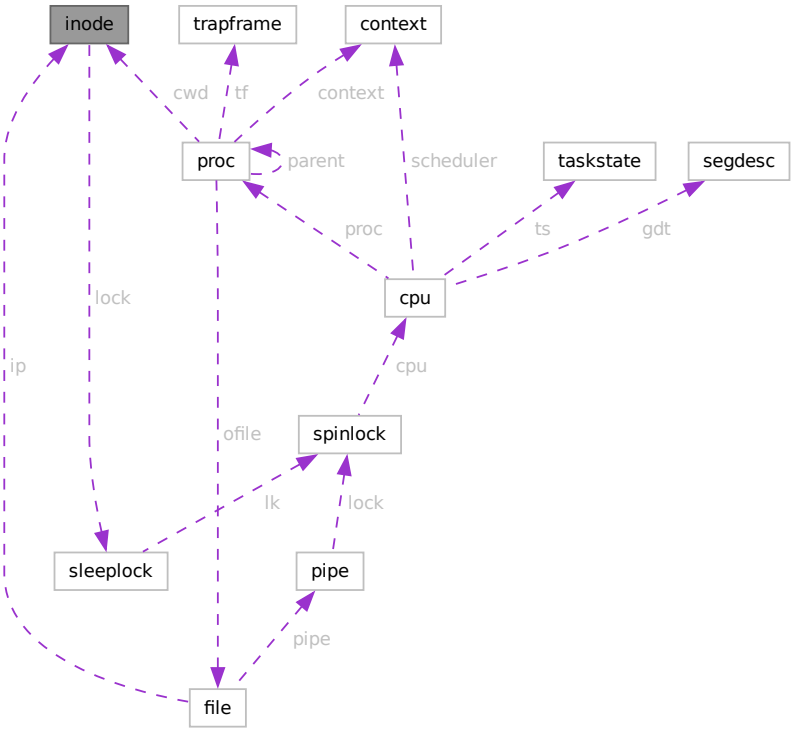
- Align **x**

The documentation for this union was generated from the following file:

- umalloc.c

3.14 inode Struct Reference

Collaboration diagram for inode:



Public Attributes

- uint **dev**
- uint **inum**
- int **ref**
- struct sleeplock **lock**
- int **valid**
- short **type**
- short **major**
- short **minor**
- short **nlink**
- uint **size**
- uint **addrs** [NDIRECT+1]

The documentation for this struct was generated from the following file:

- file.h

3.15 ioapic Struct Reference

Public Attributes

- uint **reg**
- uint **pad** [3]
- uint **data**

The documentation for this struct was generated from the following file:

- ioapic.c

3.16 listcmd Struct Reference

Collaboration diagram for listcmd:



Public Attributes

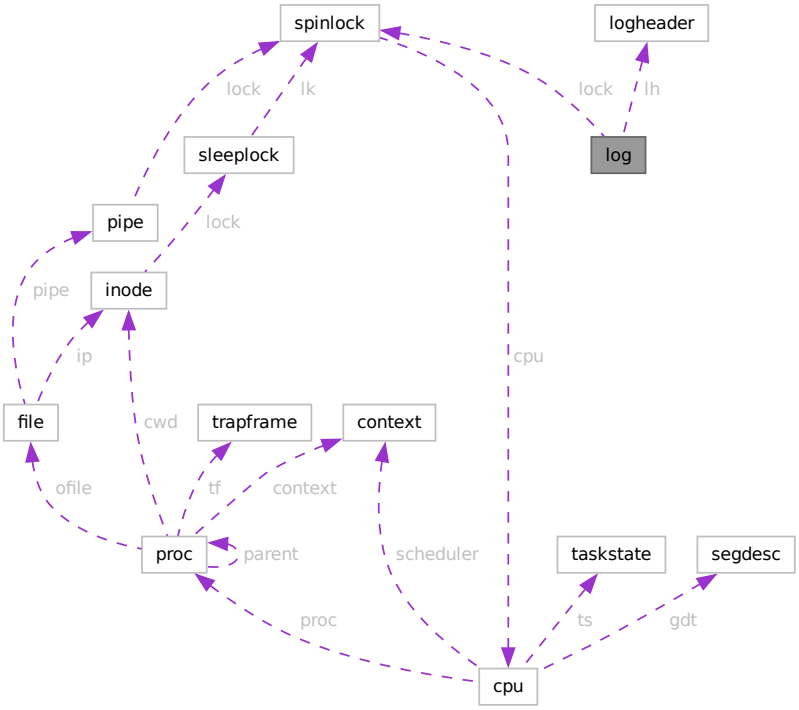
- int **type**
- struct cmd * **left**
- struct cmd * **right**

The documentation for this struct was generated from the following file:

- sh.c

3.17 log Struct Reference

Collaboration diagram for log:



Public Attributes

- struct spinlock **lock**
- int **start**
- int **size**
- int **outstanding**
- int **committing**
- int **dev**
- struct logheader **lh**

The documentation for this struct was generated from the following file:

- log.c

3.18 logheader Struct Reference

Public Attributes

- int **n**
- int **block** [LOGSIZE]

The documentation for this struct was generated from the following file:

- log.c

3.19 mp Struct Reference

Public Attributes

- uchar **signature** [4]
- void * **physaddr**
- uchar **length**
- uchar **specrev**
- uchar **checksum**
- uchar **type**
- uchar **imcrp**
- uchar **reserved** [3]

The documentation for this struct was generated from the following file:

- mp.h

3.20 mpconf Struct Reference

Public Attributes

- uchar **signature** [4]
- ushort **length**
- uchar **version**
- uchar **checksum**
- uchar **product** [20]
- uint * **oemtable**
- ushort **oemlength**
- ushort **entry**
- uint * **lapicaddr**
- ushort **xlength**
- uchar **xchecksum**
- uchar **reserved**

The documentation for this struct was generated from the following file:

- mp.h

3.21 mpioapic Struct Reference

Public Attributes

- uchar **type**
- uchar **apicno**
- uchar **version**
- uchar **flags**
- uint * **addr**

The documentation for this struct was generated from the following file:

- mp.h

3.22 mpproc Struct Reference

Public Attributes

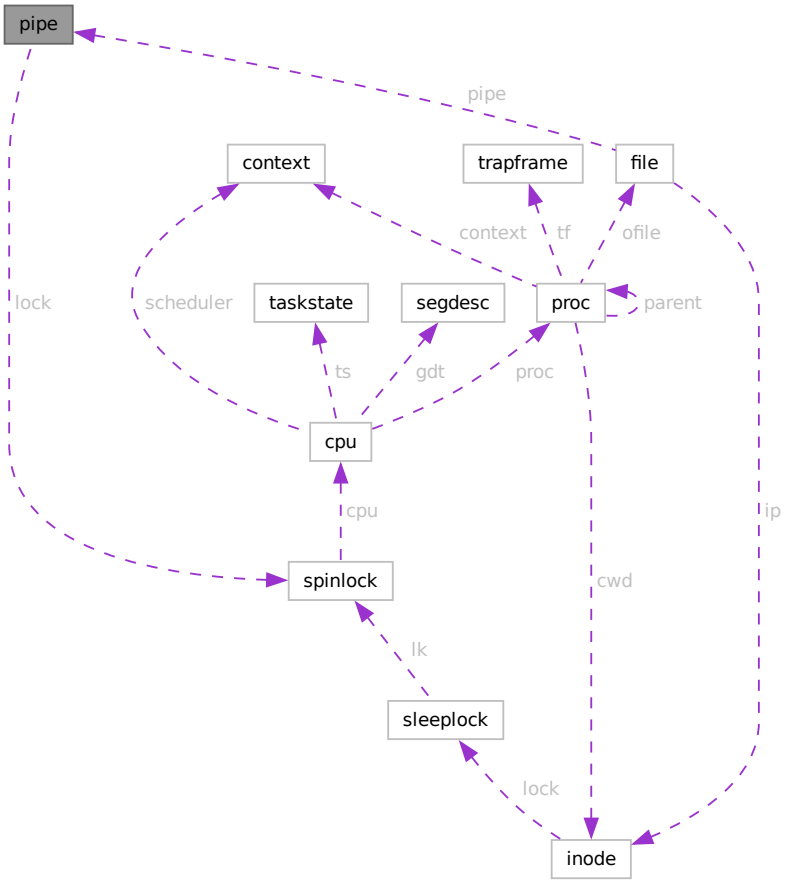
- uchar **type**
- uchar **apicid**
- uchar **version**
- uchar **flags**
- uchar **signature** [4]
- uint **feature**
- uchar **reserved** [8]

The documentation for this struct was generated from the following file:

- mp.h

3.23 pipe Struct Reference

Collaboration diagram for pipe:



Public Attributes

- struct spinlock **lock**
- char **data** [PIPESIZE]
- uint **nread**
- uint **nwrite**
- int **readopen**
- int **writeopen**

The documentation for this struct was generated from the following file:

- pipe.c

3.24 pipecmd Struct Reference

Collaboration diagram for pipecmd:



Public Attributes

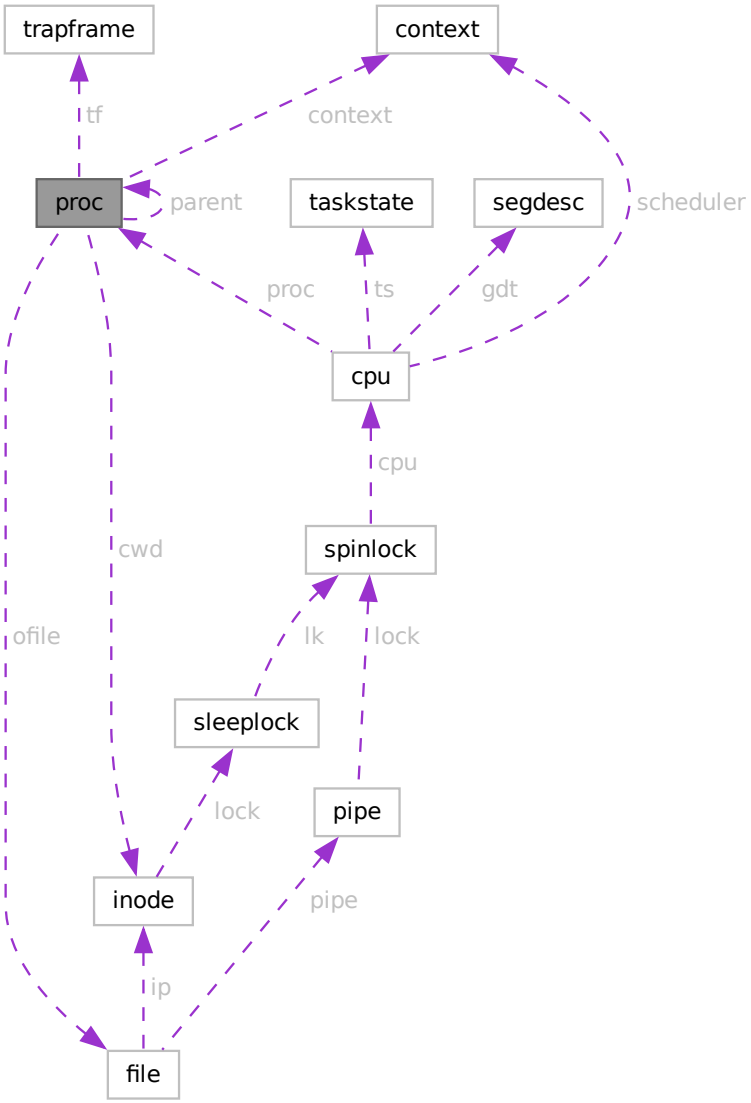
- int **type**
- struct cmd * **left**
- struct cmd * **right**

The documentation for this struct was generated from the following file:

- sh.c

3.25 proc Struct Reference

Collaboration diagram for proc:



Public Attributes

- uint **sz**
- pde_t * **pgdir**
- char * **kstack**
- enum procstate **state**
- int **pid**
- struct proc * **parent**
- struct trapframe * **tf**
- struct context * **context**

- void * **chan**
- int **killed**
- struct file * **ofile** [NOFILE]
- struct inode * **cwd**
- char **name** [16]

The documentation for this struct was generated from the following file:

- proc.h

3.26 proghdr Struct Reference

Public Attributes

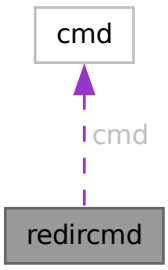
- uint **type**
- uint **off**
- uint **vaddr**
- uint **paddr**
- uint **filesz**
- uint **memsz**
- uint **flags**
- uint **align**

The documentation for this struct was generated from the following file:

- elf.h

3.27 redircmd Struct Reference

Collaboration diagram for redircmd:



Public Attributes

- int **type**
- struct cmd * **cmd**
- char * **file**
- char * **efile**
- int **mode**
- int **fd**

The documentation for this struct was generated from the following file:

- sh.c

3.28 rtcdate Struct Reference

Public Attributes

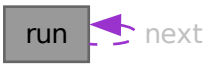
- uint **second**
- uint **minute**
- uint **hour**
- uint **day**
- uint **month**
- uint **year**

The documentation for this struct was generated from the following file:

- date.h

3.29 run Struct Reference

Collaboration diagram for run:



Public Attributes

- struct run * **next**

The documentation for this struct was generated from the following file:

- kalloc.c

3.30 segdesc Struct Reference

Public Attributes

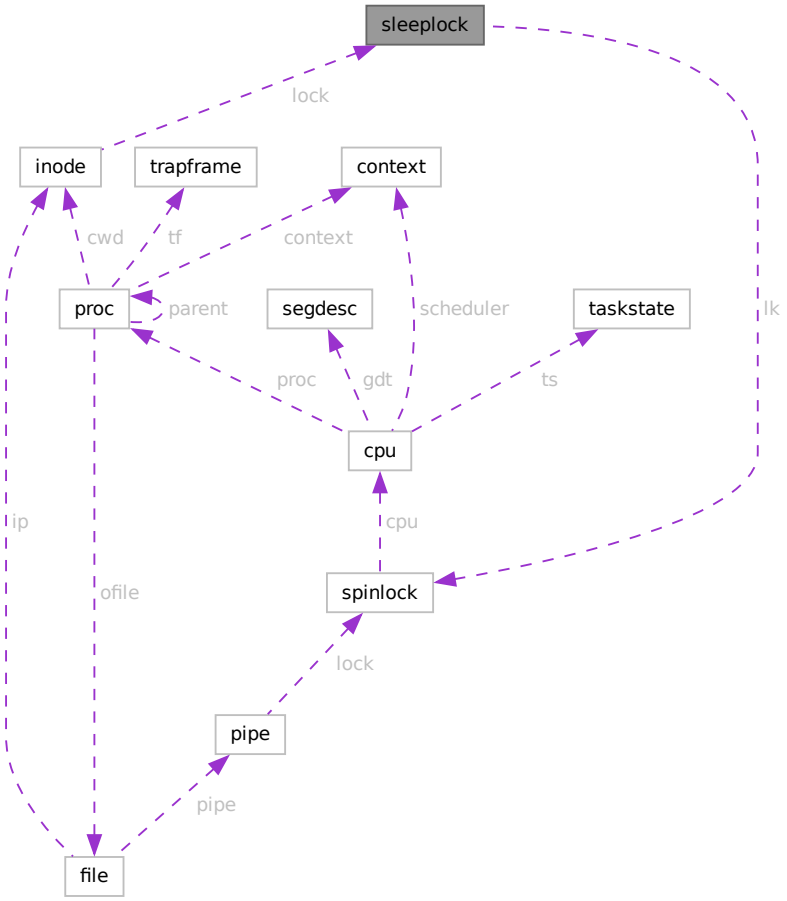
- uint **lim_15_0**: 16
- uint **base_15_0**: 16
- uint **base_23_16**: 8
- uint **type**: 4
- uint **s**: 1
- uint **dpl**: 2
- uint **p**: 1
- uint **lim_19_16**: 4
- uint **avl**: 1
- uint **rsv1**: 1
- uint **db**: 1
- uint **g**: 1
- uint **base_31_24**: 8

The documentation for this struct was generated from the following file:

- mmu.h

3.31 sleeplock Struct Reference

Collaboration diagram for sleeplock:



Public Attributes

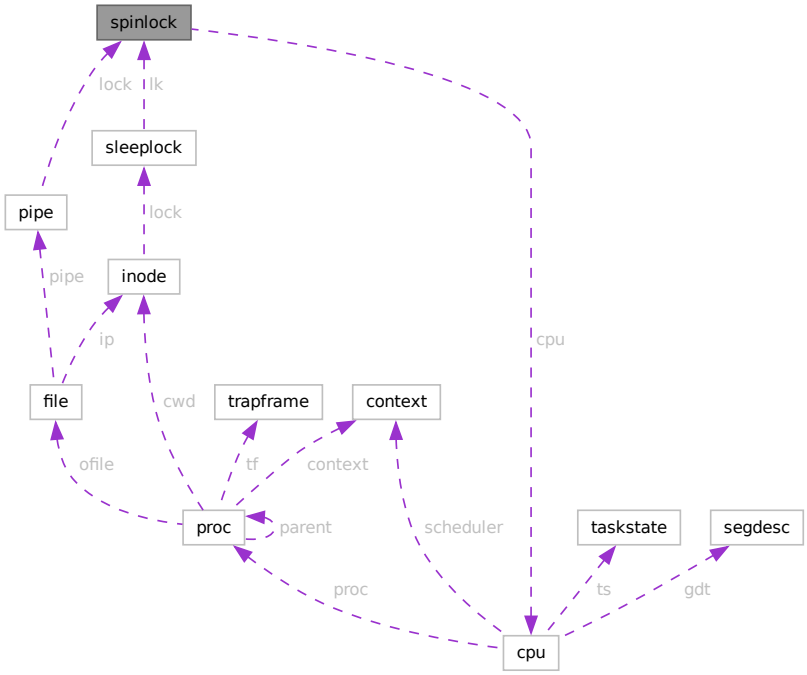
- uint **locked**
- struct spinlock **lk**
- char * **name**
- int **pid**

The documentation for this struct was generated from the following file:

- sleeplock.h

3.32 spinlock Struct Reference

Collaboration diagram for spinlock:



Public Attributes

- uint **locked**
- char * **name**
- struct cpu * **cpu**
- uint **pcs** [10]

The documentation for this struct was generated from the following file:

- spinlock.h

3.33 stat Struct Reference

Public Attributes

- short **type**
- int **dev**
- uint **ino**
- short **nlink**
- uint **size**

The documentation for this struct was generated from the following file:

- stat.h

3.34 superblock Struct Reference

Public Attributes

- uint **size**
- uint **nblocks**
- uint **ninodes**
- uint **nlog**
- uint **logstart**
- uint **inodestart**
- uint **bmapstart**

The documentation for this struct was generated from the following file:

- fs.h

3.35 taskstate Struct Reference

Public Attributes

- uint **link**
- uint **esp0**
- ushort **ss0**
- ushort **padding1**
- uint * **esp1**
- ushort **ss1**
- ushort **padding2**
- uint * **esp2**
- ushort **ss2**
- ushort **padding3**
- void * **cr3**
- uint * **eip**
- uint **eflags**
- uint **eax**
- uint **ecx**
- uint **edx**
- uint **ebx**
- uint * **esp**
- uint * **ebp**
- uint **esi**
- uint **edi**
- ushort **es**
- ushort **padding4**
- ushort **cs**
- ushort **padding5**
- ushort **ss**
- ushort **padding6**
- ushort **ds**
- ushort **padding7**
- ushort **fs**
- ushort **padding8**

- ushort **gs**
- ushort **padding9**
- ushort **ldt**
- ushort **padding10**
- ushort **t**
- ushort **iomb**

The documentation for this struct was generated from the following file:

- mmu.h

3.36 trapframe Struct Reference

Public Attributes

- uint **edi**
- uint **esi**
- uint **ebp**
- uint **oesp**
- uint **ebx**
- uint **edx**
- uint **ecx**
- uint **eax**
- ushort **gs**
- ushort **padding1**
- ushort **fs**
- ushort **padding2**
- ushort **es**
- ushort **padding3**
- ushort **ds**
- ushort **padding4**
- uint **trapno**
- uint **err**
- uint **eip**
- ushort **cs**
- ushort **padding5**
- uint **eflags**
- uint **esp**
- ushort **ss**
- ushort **padding6**

The documentation for this struct was generated from the following file:

- x86.h

Chapter 4

File Documentation

4.1 asm.h

```
00001 //
00002 // assembler macros to create x86 segments
00003 //
00004
00005 #define SEG_NULLASM                                             \
00006         .word 0, 0;                                             \
00007         .byte 0, 0, 0, 0
00008
00009 // The 0xC0 means the limit is in 4096-byte units
00010 // and (for executable segments) 32-bit mode.
00011 #define SEG_ASM(type,base,lim)                                  \
00012         .word (((lim) » 12) & 0xffff), ((base) & 0xffff);      \
00013         .byte (((base) » 16) & 0xff), (0x90 | (type)),         \
00014                 (0xC0 | (((lim) » 28) & 0xf)), (((base) » 24) & 0xff)
00015
00016 #define STA_X     0x8       // Executable segment
00017 #define STA_W     0x2       // Writeable (non-executable segments)
00018 #define STA_R     0x2       // Readable (executable segments)
```

4.2 buf.h

```
00001 struct buf {
00002   int flags;
00003   uint dev;
00004   uint blockno;
00005   struct sleeplock lock;
00006   uint refcnt;
00007   struct buf *prev; // LRU cache list
00008   struct buf *next;
00009   struct buf *qnext; // disk queue
00010   uchar data[BSIZE];
00011 };
00012 #define B_VALID 0x2  // buffer has been read from disk
00013 #define B_DIRTY 0x4  // buffer needs to be written to disk
00014
```

4.3 date.h

```
00001 struct rtcdate {
00002   uint second;
00003   uint minute;
00004   uint hour;
00005   uint day;
00006   uint month;
00007   uint year;
00008 };
```

4.4 defs.h

```
00001 struct buf;
00002 struct context;
00003 struct file;
00004 struct inode;
00005 struct pipe;
00006 struct proc;
00007 struct rtcdate;
00008 struct spinlock;
00009 struct sleeplock;
00010 struct stat;
00011 struct superblock;
00012
00013 // bio.c
00014 void            binit(void);
00015 struct buf*     bread(uint, uint);
00016 void            brelse(struct buf*);
00017 void            bwrite(struct buf*);
00018
00019 // console.c
00020 void            consoleinit(void);
00021 void            cprintf(char*, ...);
00022 void            consoleintr(int(*)(void));
00023 void            panic(char*) __attribute__((noreturn));
00024
00025 // exec.c
00026 int             exec(char*, char**);
00027
00028 // file.c
00029 struct file*    filealloc(void);
00030 void            fileclose(struct file*);
00031 struct file*    filedup(struct file*);
00032 void            fileinit(void);
00033 int             fileread(struct file*, char*, int n);
00034 int             filestat(struct file*, struct stat*);
00035 int             filewrite(struct file*, char*, int n);
00036
00037 // fs.c
00038 void            readsb(int dev, struct superblock *sb);
00039 int             dirlink(struct inode*, char*, uint);
00040 struct inode*   dirlookup(struct inode*, char*, uint*);
00041 struct inode*   ialloc(uint, short);
00042 struct inode*   idup(struct inode*);
00043 void            iinit(int dev);
00044 void            ilock(struct inode*);
00045 void            iput(struct inode*);
00046 void            iunlock(struct inode*);
00047 void            iunlockput(struct inode*);
00048 void            iupdate(struct inode*);
00049 int             namecmp(const char*, const char*);
00050 struct inode*   namei(char*);
00051 struct inode*   nameiparent(char*, char*);
00052 int             readi(struct inode*, char*, uint, uint);
00053 void            stati(struct inode*, struct stat*);
00054 int             writei(struct inode*, char*, uint, uint);
00055
00056 // ide.c
00057 void            ideinit(void);
00058 void            ideintr(void);
00059 void            iderw(struct buf*);
00060
00061 // ioapic.c
00062 void            ioapicenable(int irq, int cpu);
00063 extern uchar    ioapicid;
00064 void            ioapicinit(void);
00065
00066 // kalloc.c
00067 char*           kalloc(void);
00068 void            kfree(char*);
00069 void            kinit1(void*, void*);
00070 void            kinit2(void*, void*);
00071
00072 // kbd.c
00073 void            kbdintr(void);
00074
00075 // lapic.c
00076 void            cmostime(struct rtcdate *r);
00077 int             lapicid(void);
00078 extern volatile uint*    lapic;
00079 void            lapiceoi(void);
00080 void            lapicinit(void);
00081 void            lapicstartap(uchar, uint);
00082 void            microdelay(int);
00083
00084 // log.c
```

```
00085 void            initlog(int dev);
00086 void            log_write(struct buf*);
00087 void            begin_op();
00088 void            end_op();
00089
00090 // mp.c
00091 extern int      ismp;
00092 void            mpinit(void);
00093
00094 // picirq.c
00095 void            picenable(int);
00096 void            picinit(void);
00097
00098 // pipe.c
00099 int             pipealloc(struct file**, struct file**);
00100 void            pipeclose(struct pipe*, int);
00101 int             piperead(struct pipe*, char*, int);
00102 int             pipewrite(struct pipe*, char*, int);
00103
00104 //PAGEBREAK: 16
00105 // proc.c
00106 int             cpuid(void);
00107 void            exit(void);
00108 int             fork(void);
00109 int             growproc(int);
00110 int             kill(int);
00111 struct cpu*     mycpu(void);
00112 struct proc*    myproc();
00113 void            pinit(void);
00114 void            procdump(void);
00115 void            scheduler(void) __attribute__((noreturn));
00116 void            sched(void);
00117 void            setproc(struct proc*);
00118 void            sleep(void*, struct spinlock*);
00119 void            userinit(void);
00120 int             wait(void);
00121 void            wakeup(void*);
00122 void            yield(void);
00123
00124 // swtch.S
00125 void            swtch(struct context**, struct context*);
00126
00127 // spinlock.c
00128 void            acquire(struct spinlock*);
00129 void            getcallerpcs(void*, uint*);
00130 int             holding(struct spinlock*);
00131 void            initlock(struct spinlock*, char*);
00132 void            release(struct spinlock*);
00133 void            pushcli(void);
00134 void            popcli(void);
00135
00136 // sleeplock.c
00137 void            acquiresleep(struct sleeplock*);
00138 void            releasesleep(struct sleeplock*);
00139 int             holdingsleep(struct sleeplock*);
00140 void            initsleeplock(struct sleeplock*, char*);
00141
00142 // string.c
00143 int             memcmp(const void*, const void*, uint);
00144 void*           memmove(void*, const void*, uint);
00145 void*           memset(void*, int, uint);
00146 char*           safestrcpy(char*, const char*, int);
00147 int             strlen(const char*);
00148 int             strncmp(const char*, const char*, uint);
00149 char*           strncpy(char*, const char*, int);
00150
00151 // syscall.c
00152 int             argint(int, int*);
00153 int             argptr(int, char**, int);
00154 int             argstr(int, char**);
00155 int             fetchint(uint, int*);
00156 int             fetchstr(uint, char**);
00157 void            syscall(void);
00158
00159 // timer.c
00160 void            timerinit(void);
00161
00162 // trap.c
00163 void            idtinit(void);
00164 extern uint     ticks;
00165 void            tvinit(void);
00166 extern struct spinlock tickslock;
00167
00168 // uart.c
00169 void            uartinit(void);
00170 void            uartintr(void);
00171 void            uartputc(int);
```

```
00172
00173 // vm.c
00174 void            seginit(void);
00175 void            kvmalloc(void);
00176 pde_t*          setupkvm(void);
00177 char*           uva2ka(pde_t*, char*);
00178 int             allocuvm(pde_t*, uint, uint);
00179 int             deallocuvm(pde_t*, uint, uint);
00180 void            freevm(pde_t*);
00181 void            inituvm(pde_t*, char*, uint);
00182 int             loaduvm(pde_t*, char*, struct inode*, uint, uint);
00183 pde_t*          copyuvm(pde_t*, uint);
00184 void            switchuvm(struct proc*);
00185 void            switchkvm(void);
00186 int             copyout(pde_t*, uint, void*, uint);
00187 void            clearpteu(pde_t *pgdir, char *uva);
00188
00189 // number of elements in fixed-size array
00190 #define NELEM(x) (sizeof(x)/sizeof((x)[0]))
```

4.5 elf.h

```
00001 // Format of an ELF executable file
00002
00003 #define ELF_MAGIC 0x464C457FU  // "\x7FELF" in little endian
00004
00005 // File header
00006 struct elfhdr {
00007   uint magic;  // must equal ELF_MAGIC
00008   uchar elf[12];
00009   ushort type;
00010   ushort machine;
00011   uint version;
00012   uint entry;
00013   uint phoff;
00014   uint shoff;
00015   uint flags;
00016   ushort ehsize;
00017   ushort phentsize;
00018   ushort phnum;
00019   ushort shentsize;
00020   ushort shnum;
00021   ushort shstrndx;
00022 };
00023
00024 // Program section header
00025 struct proghdr {
00026   uint type;
00027   uint off;
00028   uint vaddr;
00029   uint paddr;
00030   uint filesz;
00031   uint memsz;
00032   uint flags;
00033   uint align;
00034 };
00035
00036 // Values for Proghdr type
00037 #define ELF_PROG_LOAD           1
00038
00039 // Flag bits for Proghdr flags
00040 #define ELF_PROG_FLAG_EXEC      1
00041 #define ELF_PROG_FLAG_WRITE     2
00042 #define ELF_PROG_FLAG_READ      4
```

4.6 fcntl.h

```
00001 #define O_RDONLY  0x000
00002 #define O_WRONLY  0x001
00003 #define O_RDWR    0x002
00004 #define O_CREATE  0x200
```

4.7 file.h

```
00001 struct file {
```

```
00002   enum { FD_NONE, FD_PIPE, FD_INODE } type;
00003   int ref; // reference count
00004   char readable;
00005   char writable;
00006   struct pipe *pipe;
00007   struct inode *ip;
00008   uint off;
00009 };
00010
00011
00012 // in-memory copy of an inode
00013 struct inode {
00014   uint dev;           // Device number
00015   uint inum;          // Inode number
00016   int ref;            // Reference count
00017   struct sleeplock lock; // protects everything below here
00018   int valid;          // inode has been read from disk?
00019
00020   short type;         // copy of disk inode
00021   short major;
00022   short minor;
00023   short nlink;
00024   uint size;
00025   uint addrs[NDIRECT+1];
00026 };
00027
00028 // table mapping major device number to
00029 // device functions
00030 struct devsw {
00031   int (*read)(struct inode*, char*, int);
00032   int (*write)(struct inode*, char*, int);
00033 };
00034
00035 extern struct devsw devsw[];
00036
00037 #define CONSOLE 1
```

4.8 fs.h

```
00001 // On-disk file system format.
00002 // Both the kernel and user programs use this header file.
00003
00004
00005 #define ROOTINO 1  // root i-number
00006 #define BSIZE 512  // block size
00007
00008 // Disk layout:
00009 // [ boot block | super block | log | inode blocks |
00010 //                                          free bit map | data blocks]
00011 //
00012 // mkfs computes the super block and builds an initial file system. The
00013 // super block describes the disk layout:
00014 struct superblock {
00015   uint size;         // Size of file system image (blocks)
00016   uint nblocks;      // Number of data blocks
00017   uint ninodes;      // Number of inodes.
00018   uint nlog;         // Number of log blocks
00019   uint logstart;     // Block number of first log block
00020   uint inodestart;   // Block number of first inode block
00021   uint bmapstart;    // Block number of first free map block
00022 };
00023
00024 #define NDIRECT 12
00025 #define NINDIRECT (BSIZE / sizeof(uint))
00026 #define MAXFILE (NDIRECT + NINDIRECT)
00027
00028 // On-disk inode structure
00029 struct dinode {
00030   short type;           // File type
00031   short major;          // Major device number (T_DEV only)
00032   short minor;          // Minor device number (T_DEV only)
00033   short nlink;          // Number of links to inode in file system
00034   uint size;            // Size of file (bytes)
00035   uint addrs[NDIRECT+1];   // Data block addresses
00036 };
00037
00038 // Inodes per block.
00039 #define IPB           (BSIZE / sizeof(struct dinode))
00040
00041 // Block containing inode i
00042 #define IBLOCK(i, sb)     ((i) / IPB + sb.inodestart)
00043
00044 // Bitmap bits per block
```

```
00045 #define BPB           (BSIZE*8)
00046
00047 // Block of free map containing bit for block b
00048 #define BBLOCK(b, sb) (b/BPB + sb.bmapstart)
00049
00050 // Directory is a file containing a sequence of dirent structures.
00051 #define DIRSIZ 14
00052
00053 struct dirent {
00054   ushort inum;
00055   char name[DIRSIZ];
00056 };
00057
```

4.9 kbd.h

```
00001 // PC keyboard interface constants
00002
00003 #define KBSTATP         0x64    // kbd controller status port(I)
00004 #define KBS_DIB         0x01    // kbd data in buffer
00005 #define KBDATAP         0x60    // kbd data port(I)
00006
00007 #define NO              0
00008
00009 #define SHIFT           (1«0)
00010 #define CTL             (1«1)
00011 #define ALT             (1«2)
00012
00013 #define CAPSLOCK        (1«3)
00014 #define NUMLOCK         (1«4)
00015 #define SCROLLLOCK      (1«5)
00016
00017 #define E0ESC           (1«6)
00018
00019 // Special keycodes
00020 #define KEY_HOME        0xE0
00021 #define KEY_END         0xE1
00022 #define KEY_UP          0xE2
00023 #define KEY_DN          0xE3
00024 #define KEY_LF          0xE4
00025 #define KEY_RT          0xE5
00026 #define KEY_PGUP        0xE6
00027 #define KEY_PGDN        0xE7
00028 #define KEY_INS         0xE8
00029 #define KEY_DEL         0xE9
00030
00031 // C('A') == Control-A
00032 #define C(x) (x - '@')
00033
00034 static uchar shiftcode[256] =
00035 {
00036   [0x1D] CTL,
00037   [0x2A] SHIFT,
00038   [0x36] SHIFT,
00039   [0x38] ALT,
00040   [0x9D] CTL,
00041   [0xB8] ALT
00042 };
00043
00044 static uchar togglecode[256] =
00045 {
00046   [0x3A] CAPSLOCK,
00047   [0x45] NUMLOCK,
00048   [0x46] SCROLLLOCK
00049 };
00050
00051 static uchar normalmap[256] =
00052 {
00053   NO,   0x1B, '1',  '2',  '3',  '4',  '5',  '6',  // 0x00
00054   '7',  '8',  '9',  '0',  '-',  '=',  '\b', '\t',
00055   'q',  'w',  'e',  'r',  't',  'y',  'u',  'i',  // 0x10
00056   'o',  'p',  '[',  ']',  '\n', NO,   'a',  's',
00057   'd',  'f',  'g',  'h',  'j',  'k',  'l',  ';',  // 0x20
00058   '\'', '`',  NO,   '\\', 'z',  'x',  'c',  'v',
00059   'b',  'n',  'm',  ',',  '.',  '/',  NO,   '*',  // 0x30
00060   NO,   ' ',  NO,   NO,   NO,   NO,   NO,   NO,
00061   NO,   NO,   NO,   NO,   NO,   NO,   NO,   '7',  // 0x40
00062   '8',  '9',  '-',  '4',  '5',  '6',  '+',  '1',
00063   '2',  '3',  '0',  '.',  NO,   NO,   NO,   NO,   // 0x50
00064   [0x9C] '\n',      // KP_Enter
00065   [0xB5] '/',       // KP_Div
00066   [0xC8] KEY_UP,    [0xD0] KEY_DN,
00067   [0xC9] KEY_PGUP,  [0xD1] KEY_PGDN,
```

```
00068   [0xCB] KEY_LF,    [0xCD] KEY_RT,
00069   [0x97] KEY_HOME,  [0xCF] KEY_END,
00070   [0xD2] KEY_INS,   [0xD3] KEY_DEL
00071 };
00072
00073 static uchar shiftmap[256] =
00074 {
00075   NO,   033,  '!',  '@',  '#',  '$',  '%',  '^',  // 0x00
00076   '&',  '*',  '(',  ')',  '_',  '+',  '\b', '\t',
00077   'Q',  'W',  'E',  'R',  'T',  'Y',  'U',  'I',  // 0x10
00078   'O',  'P',  '{',  '}',  '\n', NO,   'A',  'S',
00079   'D',  'F',  'G',  'H',  'J',  'K',  'L',  ':',  // 0x20
00080   '"',  '~',  NO,   '|',  'Z',  'X',  'C',  'V',
00081   'B',  'N',  'M',  '<',  '>',  '?',  NO,   '*',  // 0x30
00082   NO,   ' ',  NO,   NO,   NO,   NO,   NO,   NO,
00083   NO,   NO,   NO,   NO,   NO,   NO,   NO,   '7',  // 0x40
00084   '8',  '9',  '-',  '4',  '5',  '6',  '+',  '1',
00085   '2',  '3',  '0',  '.',  NO,   NO,   NO,   NO,   // 0x50
00086   [0x9C] '\n',      // KP_Enter
00087   [0xB5] '/',       // KP_Div
00088   [0xC8] KEY_UP,    [0xD0] KEY_DN,
00089   [0xC9] KEY_PGUP,  [0xD1] KEY_PGDN,
00090   [0xCB] KEY_LF,    [0xCD] KEY_RT,
00091   [0x97] KEY_HOME,  [0xCF] KEY_END,
00092   [0xD2] KEY_INS,   [0xD3] KEY_DEL
00093 };
00094
00095 static uchar ctlmap[256] =
00096 {
00097   NO,      NO,      NO,      NO,      NO,      NO,      NO,      NO,
00098   NO,      NO,      NO,      NO,      NO,      NO,      NO,      NO,
00099   C('Q'),  C('W'),  C('E'),  C('R'),  C('T'),  C('Y'),  C('U'),  C('I'),
00100   C('O'),  C('P'),  NO,      NO,      '\r',    NO,      C('A'),  C('S'),
00101   C('D'),  C('F'),  C('G'),  C('H'),  C('J'),  C('K'),  C('L'),  NO,
00102   NO,      NO,      NO,      C('\\'), C('Z'),  C('X'),  C('C'),  C('V'),
00103   C('B'),  C('N'),  C('M'),  NO,      NO,      C('/'),  NO,      NO,
00104   [0x9C] '\r',      // KP_Enter
00105   [0xB5] C('/'),    // KP_Div
00106   [0xC8] KEY_UP,    [0xD0] KEY_DN,
00107   [0xC9] KEY_PGUP,  [0xD1] KEY_PGDN,
00108   [0xCB] KEY_LF,    [0xCD] KEY_RT,
00109   [0x97] KEY_HOME,  [0xCF] KEY_END,
00110   [0xD2] KEY_INS,   [0xD3] KEY_DEL
00111 };
00112
```

4.10 memlayout.h

```
00001 // Memory layout
00002
00003 #define EXTMEM  0x100000            // Start of extended memory
00004 #define PHYSTOP 0xE000000           // Top physical memory
00005 #define DEVSPACE 0xFE000000         // Other devices are at high addresses
00006
00007 // Key addresses for address space layout (see kmap in vm.c for layout)
00008 #define KERNBASE 0x80000000         // First kernel virtual address
00009 #define KERNLINK (KERNBASE+EXTMEM)  // Address where kernel is linked
00010
00011 #define V2P(a) (((uint) (a)) - KERNBASE)
00012 #define P2V(a) ((void *)(((char *) (a)) + KERNBASE))
00013
00014 #define V2P_WO(x) ((x) - KERNBASE)    // same as V2P, but without casts
00015 #define P2V_WO(x) ((x) + KERNBASE)    // same as P2V, but without casts
```

4.11 mmu.h

```
00001 // This file contains definitions for the
00002 // x86 memory management unit (MMU).
00003
00004 // Eflags register
00005 #define FL_IF           0x00000200      // Interrupt Enable
00006
00007 // Control Register flags
00008 #define CR0_PE          0x00000001      // Protection Enable
00009 #define CR0_WP          0x00010000      // Write Protect
00010 #define CR0_PG          0x80000000      // Paging
00011
00012 #define CR4_PSE         0x00000010      // Page size extension
```

```
00013
00014 // various segment selectors.
00015 #define SEG_KCODE 1  // kernel code
00016 #define SEG_KDATA 2  // kernel data+stack
00017 #define SEG_UCODE 3  // user code
00018 #define SEG_UDATA 4  // user data+stack
00019 #define SEG_TSS   5  // this process's task state
00020
00021 // cpu->gdt[NSEGS] holds the above segments.
00022 #define NSEGS     6
00023
00024 #ifndef __ASSEMBLER__
00025 // Segment Descriptor
00026 struct segdesc {
00027   uint lim_15_0 : 16;  // Low bits of segment limit
00028   uint base_15_0 : 16; // Low bits of segment base address
00029   uint base_23_16 : 8; // Middle bits of segment base address
00030   uint type : 4;       // Segment type (see STS_ constants)
00031   uint s : 1;          // 0 = system, 1 = application
00032   uint dpl : 2;        // Descriptor Privilege Level
00033   uint p : 1;          // Present
00034   uint lim_19_16 : 4;  // High bits of segment limit
00035   uint avl : 1;        // Unused (available for software use)
00036   uint rsv1 : 1;       // Reserved
00037   uint db : 1;         // 0 = 16-bit segment, 1 = 32-bit segment
00038   uint g : 1;          // Granularity: limit scaled by 4K when set
00039   uint base_31_24 : 8; // High bits of segment base address
00040 };
00041
00042 // Normal segment
00043 #define SEG(type, base, lim, dpl) (struct segdesc)    \
00044 { ((lim) » 12) & 0xffff, (uint)(base) & 0xffff,      \
00045   ((uint)(base) » 16) & 0xff, type, 1, dpl, 1,       \
00046   (uint)(lim) » 28, 0, 0, 1, 1, (uint)(base) » 24 }
00047 #define SEG16(type, base, lim, dpl) (struct segdesc)  \
00048 { (lim) & 0xffff, (uint)(base) & 0xffff,               \
00049   ((uint)(base) » 16) & 0xff, type, 1, dpl, 1,       \
00050   (uint)(lim) » 16, 0, 0, 1, 0, (uint)(base) » 24 }
00051 #endif
00052
00053 #define DPL_USER    0x3     // User DPL
00054
00055 // Application segment type bits
00056 #define STA_X       0x8     // Executable segment
00057 #define STA_W       0x2     // Writeable (non-executable segments)
00058 #define STA_R       0x2     // Readable (executable segments)
00059
00060 // System segment type bits
00061 #define STS_T32A    0x9     // Available 32-bit TSS
00062 #define STS_IG32    0xE     // 32-bit Interrupt Gate
00063 #define STS_TG32    0xF     // 32-bit Trap Gate
00064
00065 // A virtual address 'la' has a three-part structure as follows:
00066 //
00067 // +--------10------+-------10-------+---------12----------+
00068 // | Page Directory |   Page Table   | Offset within Page  |
00069 // |      Index     |      Index     |                     |
00070 // +----------------+----------------+---------------------+
00071 //  \--- PDX(va) --/ \--- PTX(va) --/
00072
00073 // page directory index
00074 #define PDX(va)         (((uint)(va) » PDXSHIFT) & 0x3FF)
00075
00076 // page table index
00077 #define PTX(va)         (((uint)(va) » PTXSHIFT) & 0x3FF)
00078
00079 // construct virtual address from indexes and offset
00080 #define PGADDR(d, t, o) ((uint)((d) « PDXSHIFT | (t) « PTXSHIFT | (o)))
00081
00082 // Page directory and page table constants.
00083 #define NPDENTRIES      1024    // # directory entries per page directory
00084 #define NPTENTRIES      1024    // # PTEs per page table
00085 #define PGSIZE          4096    // bytes mapped by a page
00086
00087 #define PTXSHIFT        12      // offset of PTX in a linear address
00088 #define PDXSHIFT        22      // offset of PDX in a linear address
00089
00090 #define PGROUNDUP(sz)  (((sz)+PGSIZE-1) & ~(PGSIZE-1))
00091 #define PGROUNDDOWN(a) (((a)) & ~(PGSIZE-1))
00092
00093 // Page table/directory entry flags.
00094 #define PTE_P           0x001   // Present
00095 #define PTE_W           0x002   // Writeable
00096 #define PTE_U           0x004   // User
00097 #define PTE_PS          0x080   // Page Size
00098
00099 // Address in page table or page directory entry
```

```
00100 #define PTE_ADDR(pte)   ((uint)(pte) & ~0xFFF)
00101 #define PTE_FLAGS(pte)  ((uint)(pte) &  0xFFF)
00102
00103 #ifndef __ASSEMBLER__
00104 typedef uint pte_t;
00105
00106 // Task state segment format
00107 struct taskstate {
00108   uint link;         // Old ts selector
00109   uint esp0;         // Stack pointers and segment selectors
00110   ushort ss0;        //   after an increase in privilege level
00111   ushort padding1;
00112   uint *esp1;
00113   ushort ss1;
00114   ushort padding2;
00115   uint *esp2;
00116   ushort ss2;
00117   ushort padding3;
00118   void *cr3;         // Page directory base
00119   uint *eip;         // Saved state from last task switch
00120   uint eflags;
00121   uint eax;          // More saved state (registers)
00122   uint ecx;
00123   uint edx;
00124   uint ebx;
00125   uint *esp;
00126   uint *ebp;
00127   uint esi;
00128   uint edi;
00129   ushort es;         // Even more saved state (segment selectors)
00130   ushort padding4;
00131   ushort cs;
00132   ushort padding5;
00133   ushort ss;
00134   ushort padding6;
00135   ushort ds;
00136   ushort padding7;
00137   ushort fs;
00138   ushort padding8;
00139   ushort gs;
00140   ushort padding9;
00141   ushort ldt;
00142   ushort padding10;
00143   ushort t;          // Trap on task switch
00144   ushort iomb;       // I/O map base address
00145 };
00146
00147 // Gate descriptors for interrupts and traps
00148 struct gatedesc {
00149   uint off_15_0 : 16;   // low 16 bits of offset in segment
00150   uint cs : 16;         // code segment selector
00151   uint args : 5;        // # args, 0 for interrupt/trap gates
00152   uint rsv1 : 3;        // reserved(should be zero I guess)
00153   uint type : 4;        // type(STS_{IG32,TG32})
00154   uint s : 1;           // must be 0 (system)
00155   uint dpl : 2;         // descriptor(meaning new) privilege level
00156   uint p : 1;           // Present
00157   uint off_31_16 : 16;  // high bits of offset in segment
00158 };
00159
00160 // Set up a normal interrupt/trap gate descriptor.
00161 // - istrap: 1 for a trap (= exception) gate, 0 for an interrupt gate.
00162 //   interrupt gate clears FL_IF, trap gate leaves FL_IF alone
00163 // - sel: Code segment selector for interrupt/trap handler
00164 // - off: Offset in code segment for interrupt/trap handler
00165 // - dpl: Descriptor Privilege Level -
00166 //        the privilege level required for software to invoke
00167 //        this interrupt/trap gate explicitly using an int instruction.
00168 #define SETGATE(gate, istrap, sel, off, d)                \
00169 {                                                         \
00170   (gate).off_15_0 = (uint)(off) & 0xffff;                \
00171   (gate).cs = (sel);                                      \
00172   (gate).args = 0;                                        \
00173   (gate).rsv1 = 0;                                        \
00174   (gate).type = (istrap) ? STS_TG32 : STS_IG32;           \
00175   (gate).s = 0;                                           \
00176   (gate).dpl = (d);                                       \
00177   (gate).p = 1;                                           \
00178   (gate).off_31_16 = (uint)(off) >> 16;                  \
00179 }
00180
00181 #endif
```

4.12 mp.h

```
00001 // See MultiProcessor Specification Version 1.[14]
00002
00003 struct mp {             // floating pointer
00004   uchar signature[4];           // "_MP_"
00005   void *physaddr;               // phys addr of MP config table
00006   uchar length;                 // 1
00007   uchar specrev;                // [14]
00008   uchar checksum;               // all bytes must add up to 0
00009   uchar type;                   // MP system config type
00010   uchar imcrp;
00011   uchar reserved[3];
00012 };
00013
00014 struct mpconf {         // configuration table header
00015   uchar signature[4];           // "PCMP"
00016   ushort length;                // total table length
00017   uchar version;                // [14]
00018   uchar checksum;               // all bytes must add up to 0
00019   uchar product[20];            // product id
00020   uint *oemtable;               // OEM table pointer
00021   ushort oemlength;             // OEM table length
00022   ushort entry;                 // entry count
00023   uint *lapicaddr;              // address of local APIC
00024   ushort xlength;               // extended table length
00025   uchar xchecksum;              // extended table checksum
00026   uchar reserved;
00027 };
00028
00029 struct mpproc {         // processor table entry
00030   uchar type;                   // entry type (0)
00031   uchar apicid;                 // local APIC id
00032   uchar version;                // local APIC verison
00033   uchar flags;                  // CPU flags
00034     #define MPBOOT 0x02           // This proc is the bootstrap processor.
00035   uchar signature[4];           // CPU signature
00036   uint feature;                 // feature flags from CPUID instruction
00037   uchar reserved[8];
00038 };
00039
00040 struct mpioapic {       // I/O APIC table entry
00041   uchar type;                   // entry type (2)
00042   uchar apicno;                 // I/O APIC id
00043   uchar version;                // I/O APIC version
00044   uchar flags;                  // I/O APIC flags
00045   uint *addr;                  // I/O APIC address
00046 };
00047
00048 // Table entry types
00049 #define MPPROC    0x00  // One per processor
00050 #define MPBUS     0x01  // One per bus
00051 #define MPIOAPIC  0x02  // One per I/O APIC
00052 #define MPIOINTR  0x03  // One per bus interrupt source
00053 #define MPLINTR   0x04  // One per system interrupt source
00054
00055 //PAGEBREAK!
00056 // Blank page.
```

4.13 param.h

```
00001 #define NPROC        64  // maximum number of processes
00002 #define KSTACKSIZE 4096  // size of per-process kernel stack
00003 #define NCPU          8  // maximum number of CPUs
00004 #define NOFILE       16  // open files per process
00005 #define NFILE       100  // open files per system
00006 #define NINODE       50  // maximum number of active i-nodes
00007 #define NDEV         10  // maximum major device number
00008 #define ROOTDEV       1  // device number of file system root disk
00009 #define MAXARG       32  // max exec arguments
00010 #define MAXOPBLOCKS  10  // max # of blocks any FS op writes
00011 #define LOGSIZE      (MAXOPBLOCKS*3)  // max data blocks in on-disk log
00012 #define NBUF         (MAXOPBLOCKS*3)  // size of disk block cache
00013 #define FSSIZE       1000  // size of file system in blocks
00014
```

4.14 proc.h

```
00001 // Per-CPU state
```

```
00002 struct cpu {
00003   uchar apicid;                // Local APIC ID
00004   struct context *scheduler;   // swtch() here to enter scheduler
00005   struct taskstate ts;         // Used by x86 to find stack for interrupt
00006   struct segdesc gdt[NSEGS];   // x86 global descriptor table
00007   volatile uint started;       // Has the CPU started?
00008   int ncli;                    // Depth of pushcli nesting.
00009   int intena;                  // Were interrupts enabled before pushcli?
00010   struct proc *proc;           // The process running on this cpu or null
00011 };
00012
00013 extern struct cpu cpus[NCPU];
00014 extern int ncpu;
00015
00016 //PAGEBREAK: 17
00017 // Saved registers for kernel context switches.
00018 // Don't need to save all the segment registers (%cs, etc),
00019 // because they are constant across kernel contexts.
00020 // Don't need to save %eax, %ecx, %edx, because the
00021 // x86 convention is that the caller has saved them.
00022 // Contexts are stored at the bottom of the stack they
00023 // describe; the stack pointer is the address of the context.
00024 // The layout of the context matches the layout of the stack in swtch.S
00025 // at the "Switch stacks" comment. Switch doesn't save eip explicitly,
00026 // but it is on the stack and allocproc() manipulates it.
00027 struct context {
00028   uint edi;
00029   uint esi;
00030   uint ebx;
00031   uint ebp;
00032   uint eip;
00033 };
00034
00035 enum procstate { UNUSED, EMBRYO, SLEEPING, RUNNABLE, RUNNING, ZOMBIE };
00036
00037 // Per-process state
00038 struct proc {
00039   uint sz;                     // Size of process memory (bytes)
00040   pde_t* pgdir;                // Page table
00041   char *kstack;                // Bottom of kernel stack for this process
00042   enum procstate state;        // Process state
00043   int pid;                     // Process ID
00044   struct proc *parent;         // Parent process
00045   struct trapframe *tf;        // Trap frame for current syscall
00046   struct context *context;     // swtch() here to run process
00047   void *chan;                  // If non-zero, sleeping on chan
00048   int killed;                  // If non-zero, have been killed
00049   struct file *ofile[NOFILE];  // Open files
00050   struct inode *cwd;           // Current directory
00051   char name[16];               // Process name (debugging)
00052 };
00053
00054 // Process memory is laid out contiguously, low addresses first:
00055 //   text
00056 //   original data and bss
00057 //   fixed-size stack
00058 //   expandable heap
```

4.15 sleeplock.h

```
00001 // Long-term locks for processes
00002 struct sleeplock {
00003   uint locked;       // Is the lock held?
00004   struct spinlock lk; // spinlock protecting this sleep lock
00005
00006   // For debugging:
00007   char *name;        // Name of lock.
00008   int pid;           // Process holding lock
00009 };
00010
```

4.16 spinlock.h

```
00001 // Mutual exclusion lock.
00002 struct spinlock {
00003   uint locked;       // Is the lock held?
00004
00005   // For debugging:
```

```
00006   char *name;        // Name of lock.
00007   struct cpu *cpu;   // The cpu holding the lock.
00008   uint pcs[10];      // The call stack (an array of program counters)
00009                      // that locked the lock.
00010 };
00011
```

4.17 stat.h

```
00001 #define T_DIR  1   // Directory
00002 #define T_FILE 2   // File
00003 #define T_DEV  3   // Device
00004
00005 struct stat {
00006   short type;  // Type of file
00007   int dev;     // File system's disk device
00008   uint ino;    // Inode number
00009   short nlink; // Number of links to file
00010   uint size;   // Size of file in bytes
00011 };
```

4.18 syscall.h

```
00001 // System call numbers
00002 #define SYS_fork    1
00003 #define SYS_exit    2
00004 #define SYS_wait    3
00005 #define SYS_pipe    4
00006 #define SYS_read    5
00007 #define SYS_kill    6
00008 #define SYS_exec    7
00009 #define SYS_fstat   8
00010 #define SYS_chdir   9
00011 #define SYS_dup    10
00012 #define SYS_getpid 11
00013 #define SYS_sbrk   12
00014 #define SYS_sleep  13
00015 #define SYS_uptime 14
00016 #define SYS_open   15
00017 #define SYS_write  16
00018 #define SYS_mknod  17
00019 #define SYS_unlink 18
00020 #define SYS_link   19
00021 #define SYS_mkdir  20
00022 #define SYS_close  21
```

4.19 traps.h

```
00001 // x86 trap and interrupt constants.
00002
00003 // Processor-defined:
00004 #define T_DIVIDE         0      // divide error
00005 #define T_DEBUG          1      // debug exception
00006 #define T_NMI            2      // non-maskable interrupt
00007 #define T_BRKPT          3      // breakpoint
00008 #define T_OFLOW          4      // overflow
00009 #define T_BOUND          5      // bounds check
00010 #define T_ILLOP          6      // illegal opcode
00011 #define T_DEVICE         7      // device not available
00012 #define T_DBLFLT         8      // double fault
00013 // #define T_COPROC      9      // reserved (not used since 486)
00014 #define T_TSS           10      // invalid task switch segment
00015 #define T_SEGNP         11      // segment not present
00016 #define T_STACK         12      // stack exception
00017 #define T_GPFLT         13      // general protection fault
00018 #define T_PGFLT         14      // page fault
00019 // #define T_RES        15      // reserved
00020 #define T_FPERR         16      // floating point error
00021 #define T_ALIGN         17      // aligment check
00022 #define T_MCHK          18      // machine check
00023 #define T_SIMDERR       19      // SIMD floating point error
00024
00025 // These are arbitrarily chosen, but with care not to overlap
```

```
00026 // processor defined exceptions or interrupt vectors.
00027 #define T_SYSCALL       64      // system call
00028 #define T_DEFAULT      500      // catchall
00029
00030 #define T_IRQ0          32      // IRQ 0 corresponds to int T_IRQ
00031
00032 #define IRQ_TIMER        0
00033 #define IRQ_KBD          1
00034 #define IRQ_COM1         4
00035 #define IRQ_IDE         14
00036 #define IRQ_ERROR       19
00037 #define IRQ_SPURIOUS    31
00038
```

4.20 types.h

```
00001 typedef unsigned int   uint;
00002 typedef unsigned short ushort;
00003 typedef unsigned char  uchar;
00004 typedef uint pde_t;
```

4.21 user.h

```
00001 struct stat;
00002 struct rtcdate;
00003
00004 // system calls
00005 int fork(void);
00006 int exit(void) __attribute__((noreturn));
00007 int wait(void);
00008 int pipe(int*);
00009 int write(int, const void*, int);
00010 int read(int, void*, int);
00011 int close(int);
00012 int kill(int);
00013 int exec(char*, char**);
00014 int open(const char*, int);
00015 int mknod(const char*, short, short);
00016 int unlink(const char*);
00017 int fstat(int fd, struct stat*);
00018 int link(const char*, const char*);
00019 int mkdir(const char*);
00020 int chdir(const char*);
00021 int dup(int);
00022 int getpid(void);
00023 char* sbrk(int);
00024 int sleep(int);
00025 int uptime(void);
00026
00027 // ulib.c
00028 int stat(const char*, struct stat*);
00029 char* strcpy(char*, const char*);
00030 void *memmove(void*, const void*, int);
00031 char* strchr(const char*, char c);
00032 int strcmp(const char*, const char*);
00033 void printf(int, const char*, ...);
00034 char* gets(char*, int max);
00035 uint strlen(const char*);
00036 void* memset(void*, int, uint);
00037 void* malloc(uint);
00038 void free(void*);
00039 int atoi(const char*);
```

4.22 x86.h

```
00001 // Routines to let C code use special x86 instructions.
00002
00003 static inline uchar
00004 inb(ushort port)
00005 {
00006   uchar data;
00007
00008   asm volatile("in %1,%0" : "=a" (data) : "d" (port));
```

```
00009   return data;
00010 }
00011
00012 static inline void
00013 insl(int port, void *addr, int cnt)
00014 {
00015   asm volatile("cld; rep insl" :
00016                "=D" (addr), "=c" (cnt) :
00017                "d" (port), "0" (addr), "1" (cnt) :
00018                "memory", "cc");
00019 }
00020
00021 static inline void
00022 outb(ushort port, uchar data)
00023 {
00024   asm volatile("out %0,%1" : : "a" (data), "d" (port));
00025 }
00026
00027 static inline void
00028 outw(ushort port, ushort data)
00029 {
00030   asm volatile("out %0,%1" : : "a" (data), "d" (port));
00031 }
00032
00033 static inline void
00034 outsl(int port, const void *addr, int cnt)
00035 {
00036   asm volatile("cld; rep outsl" :
00037                "=S" (addr), "=c" (cnt) :
00038                "d" (port), "0" (addr), "1" (cnt) :
00039                "cc");
00040 }
00041
00042 static inline void
00043 stosb(void *addr, int data, int cnt)
00044 {
00045   asm volatile("cld; rep stosb" :
00046                "=D" (addr), "=c" (cnt) :
00047                "0" (addr), "1" (cnt), "a" (data) :
00048                "memory", "cc");
00049 }
00050
00051 static inline void
00052 stosl(void *addr, int data, int cnt)
00053 {
00054   asm volatile("cld; rep stosl" :
00055                "=D" (addr), "=c" (cnt) :
00056                "0" (addr), "1" (cnt), "a" (data) :
00057                "memory", "cc");
00058 }
00059
00060 struct segdesc;
00061
00062 static inline void
00063 lgdt(struct segdesc *p, int size)
00064 {
00065   volatile ushort pd[3];
00066
00067   pd[0] = size-1;
00068   pd[1] = (uint)p;
00069   pd[2] = (uint)p » 16;
00070
00071   asm volatile("lgdt (%0)" : : "r" (pd));
00072 }
00073
00074 struct gatedesc;
00075
00076 static inline void
00077 lidt(struct gatedesc *p, int size)
00078 {
00079   volatile ushort pd[3];
00080
00081   pd[0] = size-1;
00082   pd[1] = (uint)p;
00083   pd[2] = (uint)p » 16;
00084
00085   asm volatile("lidt (%0)" : : "r" (pd));
00086 }
00087
00088 static inline void
00089 ltr(ushort sel)
00090 {
00091   asm volatile("ltr %0" : : "r" (sel));
00092 }
00093
00094 static inline uint
00095 readeflags(void)
```

```
00096 {
00097   uint eflags;
00098   asm volatile("pushfl; popl %0" : "=r" (eflags));
00099   return eflags;
00100 }
00101
00102 static inline void
00103 loadgs(ushort v)
00104 {
00105   asm volatile("movw %0, %%gs" : : "r" (v));
00106 }
00107
00108 static inline void
00109 cli(void)
00110 {
00111   asm volatile("cli");
00112 }
00113
00114 static inline void
00115 sti(void)
00116 {
00117   asm volatile("sti");
00118 }
00119
00120 static inline uint
00121 xchg(volatile uint *addr, uint newval)
00122 {
00123   uint result;
00124
00125   // The + in "+m" denotes a read-modify-write operand.
00126   asm volatile("lock; xchgl %0, %1" :
00127                "+m" (*addr), "=a" (result) :
00128                "1" (newval) :
00129                "cc");
00130   return result;
00131 }
00132
00133 static inline uint
00134 rcr2(void)
00135 {
00136   uint val;
00137   asm volatile("movl %%cr2,%0" : "=r" (val));
00138   return val;
00139 }
00140
00141 static inline void
00142 lcr3(uint val)
00143 {
00144   asm volatile("movl %0,%%cr3" : : "r" (val));
00145 }
00146
00147 //PAGEBREAK: 36
00148 // Layout of the trap frame built on the stack by the
00149 // hardware and by trapasm.S, and passed to trap().
00150 struct trapframe {
00151   // registers as pushed by pusha
00152   uint edi;
00153   uint esi;
00154   uint ebp;
00155   uint oesp;      // useless & ignored
00156   uint ebx;
00157   uint edx;
00158   uint ecx;
00159   uint eax;
00160
00161   // rest of trap frame
00162   ushort gs;
00163   ushort padding1;
00164   ushort fs;
00165   ushort padding2;
00166   ushort es;
00167   ushort padding3;
00168   ushort ds;
00169   ushort padding4;
00170   uint trapno;
00171
00172   // below here defined by x86 hardware
00173   uint err;
00174   uint eip;
00175   ushort cs;
00176   ushort padding5;
00177   uint eflags;
00178
00179   // below here only when crossing rings, such as from user to kernel
00180   uint esp;
00181   ushort ss;
00182   ushort padding6;
```

```
00183 };
```

Index

backcmd, 5
buf, 6

cmd, 7
context, 7
cpu, 8

devsw, 9
dinode, 9
dirent, 9

elfhdr, 10
execcmd, 10

file, 11

gatedesc, 12

header, 12

inode, 13
ioapic, 14

listcmd, 14
log, 15
logheader, 15

mp, 16
mpconf, 16
mpioapic, 16
mpproc, 17

pipe, 17
pipecmd, 18
proc, 19
proghdr, 20

redircmd, 20
rtcdate, 21
run, 21

segdesc, 22
sleeplock, 23
spinlock, 24
stat, 24
superblock, 25

taskstate, 25
trapframe, 26